Structured Output

Modulo M2 · Lezione 25



Un testo, tre interessi

«Mi chiamo **Giulia Bianchi**, lavoro nel marketing, ma studio **Python** nel tempo libero e mi appassiona l'**intelligenza artificiale**; il weekend faccio spesso **arrampicata**.»

Chiediamo al modello, in linguaggio naturale: *«Estrai nome, cognome e interessi.»*

La risposta

«Il nome della persona è Giulia, il cognome è Bianchi, e i suoi interessi sono la programmazione in Python, l'intelligenza artificiale e l'arrampicata.»

> Corretta, leggibile — e una **frase**, non un dato.



Agenda

Un solo file per tutta la serata: `working/structured.py` .

	Blocco	Dove si lavora	Tempo
	Teoria: il problema, la storia, come funziona davvero, il JSON Schema, l'API universale	slide	45′
	Live coding: Pydantic, structured outputs, retry	`working/structured.py`	50′
	Esercizio guidato: il metodo per lo schema, applicato su un testo dato	`working/structured.py`	20′
	Wrap-up: recap + ponte a giovedì	slide	5′

= scrive il docente, voi replicate = scrivete voi

> Chiave OpenAI: quella di **L13–L24**, nessuna registrazione nuova.



1. Il problema e la storia



Il ciclo che non si scrive

Il passo successivo della pipeline: per ogni interesse, cercare corsi di formazione.

```
for interesse in persona.interessi:
    cerca_corsi(interesse)
```

Da una frase, questo ciclo non si scrive

Non esiste una `.interessi` da iterare, solo una stringa lunga. Un tentativo con `.split(",")` o una regex si rompe al primo cambio di forma della risposta.

Come si ottiene dal modello **esattamente** la forma di cui il codice ha bisogno?



La storia, parte 1: JSON mode

DevDay · novembre 2023

JSON mode

Un parametro forza il modello a produrre testo che è sintatticamente **JSON valido**.

Cosa NON garantisce:

- quali campi compaiono
- i tipi dei valori
- l'aderenza a uno schema

> Si "prega" il prompt di metterci i campi giusti, e si valida **lato client** sperando che non manchi nulla.



Quando JSON mode si rompe

```
import json
json.loads(risposta)
```

```
Traceback (most recent call last):
  File "structured.py", line 12, in <module>
    persona = json.loads(risposta)
json.decoder.JSONDecodeError: Expecting ',' delimiter: line 3 column 5 (char 47)
```

< 40%

adherence allo schema dichiarato su casi complessi (`gpt-4-0613` , JSON mode) — il numero che ha motivato il cambio



La storia, parte 2: Structured Outputs

6 agosto 2024

Structured Outputs

Evoluzione dichiarata della JSON mode: si passa lo **schema esatto** (via Pydantic o JSON Schema diretto) e il modello è **vincolato** a produrlo, non "invitato".

100%

di adherence (`gpt-4o-2024-08-06`) — contro il <40% della slide precedente

Non un affinamento incrementale: un salto da *"probabilmente"* a *"garantito"*.

Fonte: OpenAI, "Introducing Structured Outputs in the API".



Come funziona davvero: decodifica vincolata

Il salto 40%→100% non è un modello "più bravo per caso": è un meccanismo diverso.

1 Lo schema JSON viene tradotto in una **context-free grammar (CFG)**: una grammatica formale che descrive esattamente le sequenze valide per quello schema.

↓

2 Token per token, solo i token che restano validi secondo la grammatica sono candidabili — gli altri sono esclusi **prima** di essere scelti.

↓

3 La garanzia è **strutturale**: non è che il modello "di solito" rispetta lo schema — è che non può fisicamente scrivere un token che lo violi.

Menzionare lo schema anche nel **prompt** resta utile — aiuta il modello a scegliere contenuti sensati per ogni campo — ma è un aiuto leggero al **significato**, non il meccanismo che garantisce la **forma**: quello è la grammatica vincolata, ad ogni token.



Lo JSON Schema, per intero

```
{
  "type": "object",
  "properties": {
    "nome": {"type": "string"},
    "cognome": {"type": "string"},
    "interessi": {"type": "array", "items": {"type": "string"}}
  },
  "required": ["nome", "cognome", "interessi"],
  "additionalProperties": false
}
```

`type: object` — è un oggetto con dei campi

`properties` — un campo per ognuno, col proprio tipo

`interessi` è `array` di `string`, non una stringa sola

`required` — tutti e tre obbligatori

Questo È lo schema che "specifica" — non un suggerimento, il contratto esatto passato al meccanismo appena visto.



Deep dive: i tipi di campo di uno JSON Schema

Tipo	Esempio	Quando si usa
`string`	`"nome": {"type": "string"}`	testo libero: nomi, descrizioni
`integer` / `number`	`"eta": {"type": "integer"}`	quantità, conteggi, misure
`boolean`	`"attivo": {"type": "boolean"}`	vero/falso
`array`	`"tag": {"type": "array", "items": {"type": "string"}}`	liste di elementi
`object` (annidato)	`"indirizzo": {"type": "object", "properties": {…}}`	dati raggruppati dentro un campo
`enum`	`"stato": {"enum": ["attivo", "sospeso", "chiuso"]}`	insieme CHIUSO di valori — il modello non può inventarne un quarto

L' `enum` è il tipo che userà davvero l'esercizio di stasera: l'esposizione alla luce di una pianta ha un insieme piccolo e ricorrente di valori — candidato perfetto.



Pydantic vs JSON Schema

Gli SDK accettano entrambi — perché il primo si traduce nel secondo.

```python
class Persona(BaseModel):
    nome: str
    cognome: str
```

si traduce in ↓

```json
{"nome": {"type": "string"}, "cognome": {"type": "string"}}
```

Non si sta scegliendo un meccanismo diverso: si sta scegliendo una sintassi più comoda (Python) per descrivere lo **stesso contratto**.



L’API universale

Fino a stasera, un modello sembrava avere due facce separate.

Sa cose
conoscenza, linguaggio, ragionamento

Forma dichiarata
vincolata a livello di campo

Conoscenza del modello + forma esatta dell'output

= si scrive codice che chiede al modello esattamente il dato che serve

Come a un'API tradizionale — ma capendo il linguaggio naturale dietro.



Due usi dello stesso schema

Lo stesso meccanismo (schema → grammatica → decodifica vincolata) serve in due contesti diversi.

stasera

Estrazione

Il modello **risponde** con un oggetto che rispetta lo schema: si trasforma testo in dati.

si vede più avanti · L34-L35

Tool calling

Lo schema descrive gli **argomenti** di una funzione che il modello può "invocare". Il modello non esegue nulla: l'esecuzione è un altro pezzo di codice.

> Si nomina qui solo perché lo incontrerete nella documentazione: stessa grammatica sotto, uso diverso sopra.



2. AI codice



Live coding

Si guarda e si replica, in due tappe

`working/structured.py` · 50 minuti

si torna alle slide
per il payoff

`Tappa 1 · ~22'`

Pydantic + structured outputs

Il modello `PersonaEstratta`, la prima chiamata vincolata, e `model_dump_json()`.

`Tappa 2 · ~18'`

Retry su `ValidationError`

Cosa fare quando qualcosa va storto lungo la catena.



AL RIENTRO

Tappa 1 — l'oggetto, e la stringa dietro

```python
class PersonaEstratta(BaseModel):
    nome: str
    cognome: str
    interessi: list[str]
```

Si passa `PersonaEstratta` come `response_format` : la risposta è già un'istanza validata.

```
PersonaEstratta(nome='Giulia', cognome='Bianchi', interessi=['Python', 'intelligenza artificiale', 'arrampicata'])
```

`persona.model_dump_json()` — la stringa JSON vera e propria:

```
'{"nome": "Giulia", "cognome": "Bianchi", "interessi": ["Python", "intelligenza artificiale", "arrampicata"]}'
```

Il Pydantic object è comodo per il **vostro** codice — ma è questa stringa, non un oggetto Python, quella su cui il modello lavora davvero.



AL RIENTRO

Tappa 2 — quando qualcosa va storto

Anche con lo schema garantito, la chiamata può fallire per altri motivi: rete, timeout, un caso limite.

```
try:
    persona = estrai_persona(testo)
except ValidationError:
    persona = estrai_persona(testo)  # un secondo tentativo, non un loop
```

Codice difensivo come **buona abitudine**, non come sfiducia nella garanzia di Structured Outputs.



Il payoff: il ciclo, finalmente scrivibile

```python
for interesse in persona.interessi:
    cerca_corsi(interesse)
```

```
cerco corsi su: Python
cerco corsi su: intelligenza artificiale
cerco corsi su: arrampicata
```

Il cerchio dell'esempio guida si chiude — non a parole, a schermo.



Suggerimento: come costruire uno JSON Schema

DEFINIZIONE	ESECUZIONE (SUL BOTANICO)
1 **Scrivi il JSON coi valori veri**, non i tipi — presi dal testo.	`{"nome": "Marco", "cognome": "Ferretti", "anni_esperienza": 15}`
2 **Sostituisci ogni valore col suo tipo**.	`nome → string · cognome → string · anni_esperienza → integer`
3 **Dove i valori sono un insieme chiuso e ricorrente**, usa `enum` .	`"esposizione_luce": {"enum": ["pieno sole", "luce indiretta", "ombra"]}`
4 **Per le liste di oggetti**, ripeti su un elemento poi generalizza a `array` .	`"piante": {"type": "array", "items": {"type": "object", …}}`

> Non è un obbligo rigido da seguire alla lettera: è un modo per arrivare a uno schema JSON **semanticamente sensato**, partendo da un esempio concreto invece che da un'astrazione a freddo.



Esercizio guidato

TOCCA A VOI

`working/structured.py` · 20 minuti

si torna alle slide
per il wrap-up

«Mi chiamo Marco Ferretti e lavoro come botanico da quindici anni, all'Orto Botanico di Padova. A casa ho un piccolo giardino: un Ficus lyrata in salotto, vicino alla finestra, in pieno sole, che innaffio ogni sette giorni; una Monstera deliciosa che preferisce la luce indiretta e va bagnata ogni dieci giorni; un cactus Echinocactus grusonii, in pieno sole, che innaffio molto di rado — ogni trenta giorni circa.»

Applicate il metodo appena visto: uno schema Pydantic con tutti i dati della persona e la lista delle piante — un campo `enum` incluso.

Insight: provate a estrarre anche **più campi** di quelli qui proposti — purché seguano tutti lo **stesso schema dichiarato**, e i campi che potrebbero mancare dal testo siano `nullable` (opzionali), non obbligatori a tutti i costi.



AL RIENTRO

Fatto quando

L'oggetto Python stampato a schermo ha **tutte e tre le piante**, coi campi giusti — incluso l'`enum` dell'esposizione alla luce rispettato, non tre stringhe diverse a caso.



Dal testo libero al dato

Una frase corretta ma inutile → uno schema dichiarato → un oggetto Python coi campi giusti. **Structured Outputs** fa da ponte **garantito** — e ora sapete anche *perché* lo è, non solo che lo è.

Il modulo `working/structured.py` — schema, chiamata, retry — resta come pattern riusabile.



Ponte a giovedì

> Lo stesso schema Pydantic e lo stesso pattern di chiamata torneranno su testo **vero**, preso da documenti — fatture, CV, email — invece che scritto per l'occasione.



Grazie

Giovedì — Extraction & Parsing

